Impact of LLC Replacement Policies on CPU-Based LLM Decoding

CS683 Advanced Computer Architecture

Kumar Rajnish (25M2123)

Qwen2.5-0.5B-Instruct’s Decode Workloads

- Streaming weights: with ~1GB reuse distance
- Repetitive structure: each decode step is about 280M instructions
- Small reusable data (activations, KV cache) competing for the LLC

Trace Generation

- **Workload:** Qwen2.5-0.5B-Instruct(fp16), run with llama.cpp on CPU, single thread
- **Tool:** Intel Pin 4.1 with Champsim tracer
- **Decode only:** tracing starts only at decoding phase
- **Result:** ~3.1B instructions (about 280M per decode step)

Baseline Configuration

CACHE	CONFIGURATION
L1 I-CACHE	**64 SET, 8 WAY**
L1 D-CACHE	**64 SET, 12 WAY**
L2 CACHE	**1024 SET, 8 WAY**
LAST LEVEL CACHE (LLC)	**2048 SET, 16 WAY**

No Prefetcher for all cache levels

Policy Benchmarked

- Least Recently Used (LRU) (Baseline)
- SRRIP
- Hawkeye
- Mockingjay

Hawkeye: https://github.com/Sacusa/ChampSim/tree/master
Mockingjay: https://github.com/ishanashah/Mockingjay

Baseline Results

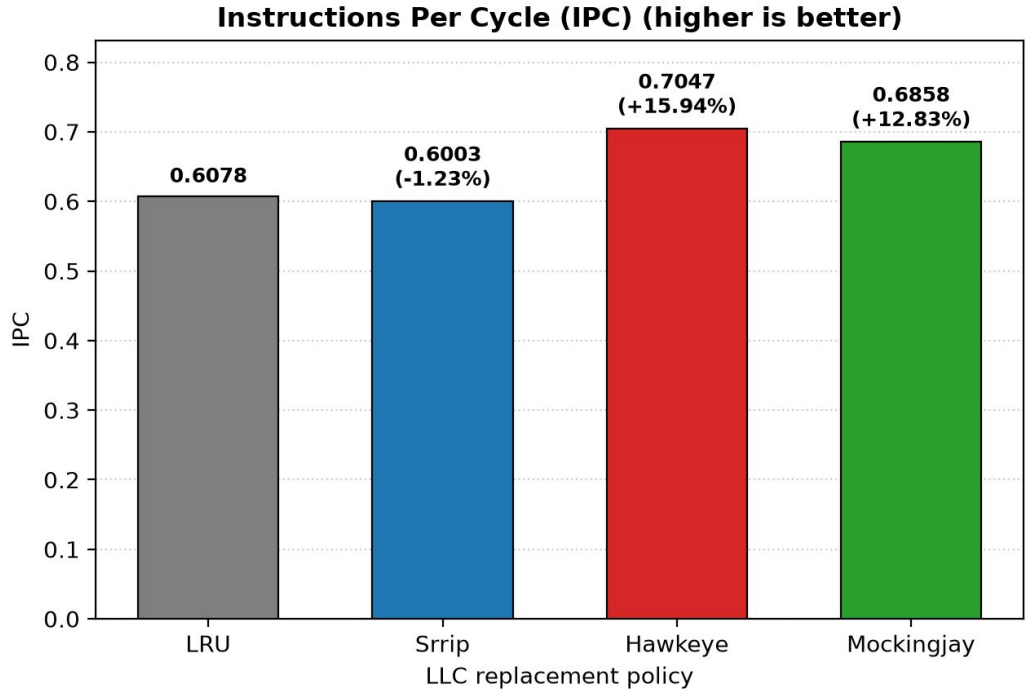
Warmup instructions: 580M (2 decode tokens)

Simulation instructions: 1.405B (5 decode tokens)

Policy: LRU

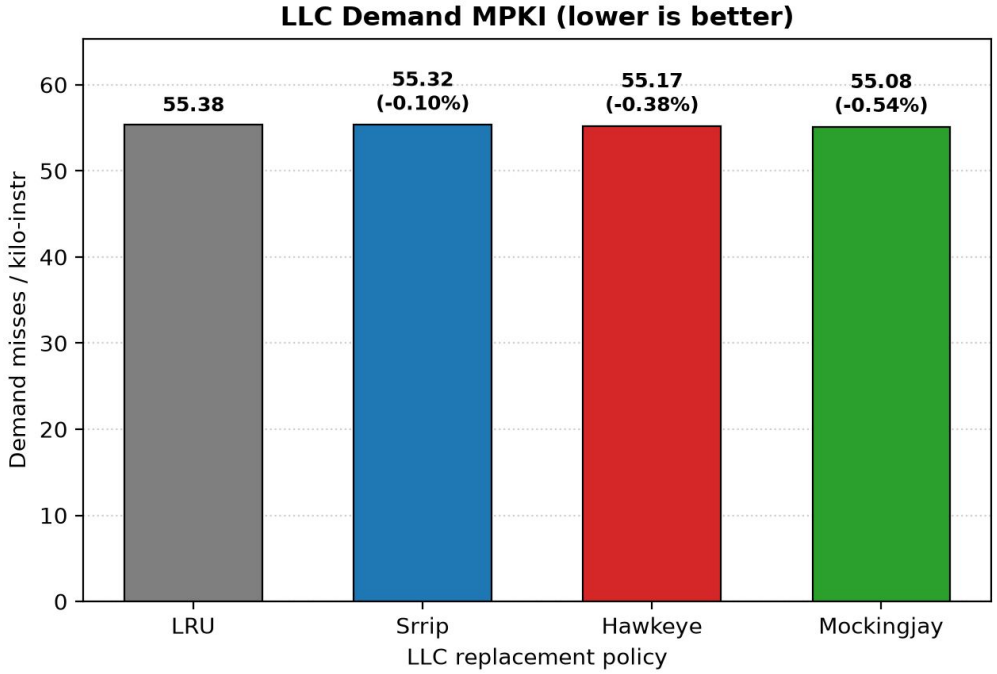
- L1I Miss Rate: 0.006%
- L1D Miss Rate: 35.59%
- L2C Miss Rate: 97.28%
- LLC Miss Rate: 99.23%
- IPC : 0.6078

Instructions Per Cycle (IPC)



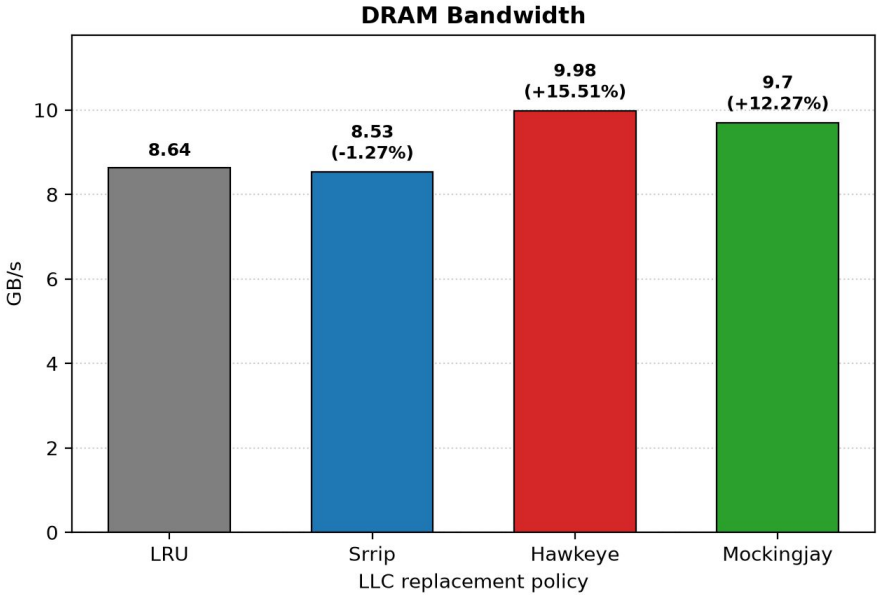
Hawakeye(+15.94%) and Mockingjay (+12.83%)

Misses Per Kilo-Instructions (MPKI)



All Replacement policy have same MPKI

If similar MPKI, then Why Speedup?



- Hawkeye and Mockingjay increases DRAM bandwidth
- possibly due to higher row-buffer hit rates, reducing average miss latency. (under investigation)

Future Plan

- Explain the Hawkeye and Mockingjay speedup by studying DRAM row-buffer access behavior
- Perform the experiment at larger LLC Cache size
- Perform the experiment at bigger decoding context length